

Homework Solutions

Question 1

(10 pts) Consider this (toy) biological setup: a cell can be in one of two states— H for high GC-content, and L for low GC. On each time step the cell produces one nucleotide (A , C , T , or G), and might also change its state. The probability of changing from state H to L is 0.5, and from state L to H is 0.4.

In state H the probabilities for producing nucleotides are 0.2 for A , 0.3 for C , 0.3 for G , and 0.2 for T . In L the probabilities are 0.3 for A , 0.2 for C , 0.2 for G , and 0.3 for T .

Consider the nucleotide sequence $S = ACCGTGCA$. Use the Viterbi algorithm to find the best state sequence and calculate the probability of S given this state sequence. Assume the previous state before S was H .

Solution

we are give the set up:

- Transitions: $P(H \rightarrow H) = 0.5$, $P(H \rightarrow L) = 0.5$, $P(L \rightarrow H) = 0.4$, $P(L \rightarrow L) = 0.6$
- Emissions in H : $A = 0.2$, $C = 0.3$, $G = 0.3$, $T = 0.2$
- Emissions in L : $A = 0.3$, $C = 0.2$, $G = 0.2$, $T = 0.3$
- Previous state (before S): H

we need to find the best path using the viterbi algorithm with the recursive formula:

$$\pi(k, v) = \max_{u \in S_{k-1}} \left(\pi(k-1, u) \cdot q(v | u) \right) \cdot e(x_k | v)$$

with $\pi(0, *) = 1$.

We now fill in the dynamic programming table. Each row corresponds to a position k in the sequence, and the columns $\pi(k, H)$ and $\pi(k, L)$ give the maximum probability of any path ending in state H or L at position k . The “from” columns record the back-pointer $\text{bp}(k, v) = \arg \max_u \pi(k-1, u) \cdot q(v | u)$, i.e., the previous state that achieved the maximum.

k	x_k	$\pi(k, H)$	bp	$\pi(k, L)$	bp
1	A	0.1000	H	0.1500	H
2	C	0.0180	L	0.0180	L
3	C	0.00270	H	0.00216	L
4	G	4.05×10^{-4}	H	2.70×10^{-4}	H
5	T	4.05×10^{-5}	H	6.075×10^{-5}	H
6	G	7.29×10^{-6}	L	7.29×10^{-6}	L
7	C	1.0935×10^{-6}	H	8.748×10^{-7}	L
8	A	1.0935×10^{-7}	H	1.64025×10^{-7}	H

We now show the full computation for each position $k = 1, \dots, 8$. At each step we apply

$$\pi(k, v) = \max_{u \in \{H, L\}} \left(\pi(k-1, u) \cdot q(v | u) \right) \cdot e(x_k | v),$$

with initialization $\pi(0, H) = 1$, $\pi(0, L) = 0$ (the previous state is H).

Position $k = 1$, $x_1 = A$:

$$\pi(1, H) = \pi(0, H) \cdot q(H | H) \cdot e(A | H) = 1 \cdot 0.5 \cdot 0.2 = 0.1000 \quad (\text{from } H)$$

$$\pi(1, L) = \pi(0, H) \cdot q(L | H) \cdot e(A | L) = 1 \cdot 0.5 \cdot 0.3 = 0.1500 \quad (\text{from } H)$$

Position $k = 2$, $x_2 = C$:

$$\pi(2, H) = \max(0.10 \cdot 0.5, 0.15 \cdot 0.4) \cdot 0.3 = \max(0.05, 0.06) \cdot 0.3 = 0.0180 \quad (\text{from } L)$$

$$\pi(2, L) = \max(0.10 \cdot 0.5, 0.15 \cdot 0.6) \cdot 0.2 = \max(0.05, 0.09) \cdot 0.2 = 0.0180 \quad (\text{from } L)$$

Position $k = 3$, $x_3 = C$:

$$\pi(3, H) = \max(0.018 \cdot 0.5, 0.018 \cdot 0.4) \cdot 0.3 = \max(0.0090, 0.0072) \cdot 0.3 = 0.00270 \quad (\text{from } H)$$

$$\pi(3, L) = \max(0.018 \cdot 0.5, 0.018 \cdot 0.6) \cdot 0.2 = \max(0.0090, 0.0108) \cdot 0.2 = 0.00216 \quad (\text{from } L)$$

Position $k = 4$, $x_4 = G$:

$$\begin{aligned} \pi(4, H) &= \max(0.00270 \cdot 0.5, 0.00216 \cdot 0.4) \cdot 0.3 = \max(0.00135, 0.000864) \cdot 0.3 \\ &= 4.05 \times 10^{-4} \quad (\text{from } H) \end{aligned}$$

$$\begin{aligned} \pi(4, L) &= \max(0.00270 \cdot 0.5, 0.00216 \cdot 0.6) \cdot 0.2 = \max(0.00135, 0.001296) \cdot 0.2 \\ &= 2.70 \times 10^{-4} \quad (\text{from } H) \end{aligned}$$

Position $k = 5$, $x_5 = T$:

$$\begin{aligned} \pi(5, H) &= \max(4.05 \times 10^{-4} \cdot 0.5, 2.70 \times 10^{-4} \cdot 0.4) \cdot 0.2 \\ &= \max(2.025 \times 10^{-4}, 1.08 \times 10^{-4}) \cdot 0.2 = 4.05 \times 10^{-5} \quad (\text{from } H) \end{aligned}$$

$$\begin{aligned} \pi(5, L) &= \max(4.05 \times 10^{-4} \cdot 0.5, 2.70 \times 10^{-4} \cdot 0.6) \cdot 0.3 \\ &= \max(2.025 \times 10^{-4}, 1.62 \times 10^{-4}) \cdot 0.3 = 6.075 \times 10^{-5} \quad (\text{from } H) \end{aligned}$$

Position $k = 6$, $x_6 = G$:

$$\begin{aligned}\pi(6, H) &= \max(4.05 \times 10^{-5} \cdot 0.5, 6.075 \times 10^{-5} \cdot 0.4) \cdot 0.3 \\ &= \max(2.025 \times 10^{-5}, 2.43 \times 10^{-5}) \cdot 0.3 = 7.29 \times 10^{-6} \quad (\text{from } L) \\ \pi(6, L) &= \max(4.05 \times 10^{-5} \cdot 0.5, 6.075 \times 10^{-5} \cdot 0.6) \cdot 0.2 \\ &= \max(2.025 \times 10^{-5}, 3.645 \times 10^{-5}) \cdot 0.2 = 7.29 \times 10^{-6} \quad (\text{from } L)\end{aligned}$$

Position $k = 7$, $x_7 = C$:

$$\begin{aligned}\pi(7, H) &= \max(7.29 \times 10^{-6} \cdot 0.5, 7.29 \times 10^{-6} \cdot 0.4) \cdot 0.3 \\ &= \max(3.645 \times 10^{-6}, 2.916 \times 10^{-6}) \cdot 0.3 = 1.0935 \times 10^{-6} \quad (\text{from } H) \\ \pi(7, L) &= \max(7.29 \times 10^{-6} \cdot 0.5, 7.29 \times 10^{-6} \cdot 0.6) \cdot 0.2 \\ &= \max(3.645 \times 10^{-6}, 4.374 \times 10^{-6}) \cdot 0.2 = 8.748 \times 10^{-7} \quad (\text{from } L)\end{aligned}$$

Position $k = 8$, $x_8 = A$:

$$\begin{aligned}\pi(8, H) &= \max(1.0935 \times 10^{-6} \cdot 0.5, 8.748 \times 10^{-7} \cdot 0.4) \cdot 0.2 \\ &= \max(5.4675 \times 10^{-7}, 3.4992 \times 10^{-7}) \cdot 0.2 = 1.0935 \times 10^{-7} \quad (\text{from } H) \\ \pi(8, L) &= \max(1.0935 \times 10^{-6} \cdot 0.5, 8.748 \times 10^{-7} \cdot 0.6) \cdot 0.3 \\ &= \max(5.4675 \times 10^{-7}, 5.2488 \times 10^{-7}) \cdot 0.3 = 1.64025 \times 10^{-7} \quad (\text{from } H)\end{aligned}$$

Once the table is complete, we recover the optimal state sequence by tracing the back-pointers from the final column. The maximum value is $\pi(8, L) = 1.64025 \times 10^{-7}$, whose back-pointer leads to H at $k = 7$; continuing backward through the bp column yields:

k	1	2	3	4	5	6	7	8
x_k	A	C	C	G	T	G	C	A
y_k	H	L	H	H	L	H	H	L

Question 2

(10 pts) In class we saw the trigram HMM model and the corresponding Viterbi algorithm. We will now make two main changes. First, we will consider a four-gram tagger, where p takes the form:

$$p(x_1 \cdots x_n, y_1 \cdots y_{n+1}) = \prod_{i=1}^{n+1} q(y_i \mid y_{i-3}, y_{i-2}, y_{i-1}) \prod_{i=1}^n e(x_i \mid y_i) \quad (1)$$

We assume in this definition that $y_0 = y_{-1} = y_{-2} = *$, where $*$ is the START symbol, $y_{n+1} = \text{STOP}$, and $y_i \in \mathcal{K}$ for $i = 1 \cdots n$, where $\mathcal{K}$ is the set of possible tags in the HMM.

Second, we consider a version of the Viterbi algorithm that takes as input an integer n (and not a sentence $x_1 \cdots x_n$ as we saw in class) and finds

$$\max_{y_1 \cdots y_{n+1}, x_1 \cdots x_n} p(x_1 \cdots x_n, y_1 \cdots y_{n+1})$$

for a four-gram tagger, as defined in Equation 1. $x_1 \cdots x_n$ may range over the values of some fixed vocabulary V . Complete the following pseudo-code of this version of the Viterbi algorithm for this model. The pseudo-code must be efficient.

Solution

Since the input is only the length n , we must jointly maximize over both the tag sequence and the word sequence. The crucial structural fact is that in Equation 1, each word x_i appears in exactly one factor, $e(x_i | y_i)$, so it has no interaction with any other word or with any tag other than its own. So once a tag y_i is chosen, the best word for that position is independent of every other choice. Define:

$$e^*(y) := \max_{x \in V} e(x | y), \quad \text{so that} \quad \max_{x_i \in V} e(x_i | y_i) = e^*(y_i).$$

This means the algorithm effectively just predicts a sequence of *tags*, and for each chosen tag it independently emits the most likely word of that category: the most likely noun, the most likely verb, the most likely adjective, and so on. There is no coupling between words: the resulting sentence is the concatenation of each category's single most probable filler, regardless of whether those fillers form a plausible sentence together. For instance, the output could easily be something like “the dog goes to work” which is syntactically fine, since each word is the most probable filler for its tag, but the model has no mechanism to enforce that the words likely to come together.

Operationally, we precompute $e^*(y)$ for every tag $y \in \mathcal{K}$ in $O(|\mathcal{K}| \cdot |V|)$ time. The problem then reduces to running four-gram Viterbi over tags only, replacing $e(x_i | y_i)$ by the constant $e^*(y_i)$.

Now, let

$$\pi(i, u, v, w) = \max_{y_1 \dots y_i : y_{i-2}=u, y_{i-1}=v, y_i=w} \prod_{j=1}^i q(y_j | y_{j-3}, y_{j-2}, y_{j-1}) \cdot e^*(y_j).$$

That is, the best score for a length- i tag prefix ending in the triple (u, v, w) , with the optimal word chosen at each position.

Pseudo-code

Input: integer n , tag set $\mathcal{K}$, vocabulary V , parameters $q(\cdot | \cdot, \cdot, \cdot)$ and $e(\cdot | \cdot)$

Precompute:

for each $y \in \mathcal{K}$: $e^*(y) \leftarrow \max_{x \in V} e(x | y)$

Initialization:

$\pi(0, *, *, *) \leftarrow 1$

$\pi(i, u, v, w) \leftarrow 0$ for all other entries

Recurrence: for $i = 1, 2, \dots, n$:

for all $u, v, w \in \mathcal{K}_{i-2} \times \mathcal{K}_{i-1} \times \mathcal{K}_i$:

$$\pi(i, u, v, w) \leftarrow \max_{t \in \mathcal{K}_{i-3}} \left(\pi(i-1, t, u, v) \cdot q(w | t, u, v) \cdot e^*(w) \right)$$

Termination:

return $\max_{u, v, w \in \mathcal{K}} \pi(n, u, v, w) \cdot q(\text{STOP} | u, v, w)$

where $\mathcal{K}_{-2} = \mathcal{K}_{-1} = \mathcal{K}_0 = \{*\}$ and $\mathcal{K}_i = \mathcal{K}$ for $i \geq 1$.

Complexity

We break the cost into three parts: precomputing e^* , the size of the DP table, and the work per cell.

For each tag $y \in \mathcal{K}$ we scan the vocabulary once to find $\max_{x \in V} e(x \mid y)$, which costs $|\mathcal{K}| \cdot |V| = O(|\mathcal{K}| \cdot |V|)$. This is done once before the DP begins.

The DP table is indexed by (i, u, v, w) , where i ranges over n positions and each of u, v, w ranges over $\mathcal{K}$. We need a cell per triple because the optimal path could end with any last-three-tag combination, giving $n \cdot |\mathcal{K}|^3$ cells.

Filling each cell requires a maximization over $t \in \mathcal{K}$:

$$\pi(i, u, v, w) = \max_{t \in \mathcal{K}} \left(\pi(i-1, t, u, v) \cdot q(w \mid t, u, v) \cdot e^*(w) \right).$$

A single pass with a running max gives $O(|\mathcal{K}|)$ per cell. We cannot reuse this max across cells because $q(w \mid t, u, v)$ depends on t .

Multiplying cells by per-cell work and adding the precomputation:

$$O(|\mathcal{K}| \cdot |V|) + \underbrace{n \cdot |\mathcal{K}|^3}_{\text{cells}} \cdot \underbrace{|\mathcal{K}|}_{\text{per cell}} = O(|\mathcal{K}| \cdot |V|) + O(n \cdot |\mathcal{K}|^4).$$

Note that a naive approach which chose the best word *inside* the DP would maximize over both $t \in \mathcal{K}$ and $x \in V$ at every cell, giving $O(n \cdot |\mathcal{K}|^4 \cdot |V|)$. Precomputing e^* pulls the $|V|$ factor out of the inner loop and pays it only once additively, which is the source of the efficiency gain. Also notice that this matches the cost of a standard 4gram Viterbi where the words are given: the $e^*(w)$ factor inside the DP is just a precomputed constant lookup, no different from looking up $e(x_i \mid w)$ when x_i is provided. The $|V|$ work was already paid once additively in the precomputation, so it does not multiply into the inner loop.

To recover the optimal sentence and tagging, we store back-pointers $\text{bp}(i, u, v, w) = \arg \max_t (\pi(i-1, t, u, v) \cdot q(w \mid t, u, v))$ during the recurrence. After termination, we trace back through the back-pointers to recover the tag sequence $y_1, \dots, y_n$, and then set

$$x_i = \arg \max_{x \in V} e(x \mid y_i)$$

for each i to obtain the corresponding words.